

TraceProv: All You Need is Log - Efficient Bolt-on Provenance Capture for Complex Relational Queries

Vinayak Jha
University of Illinois, Chicago
vinayakj@uic.edu

Boris Glavic
University of Illinois, Chicago
bglavic@uic.edu

ABSTRACT

A long line of work in data provenance has investigated how to add support for provenance capture to relational databases. However, achieving high performance for complex queries has been challenging: systems that have low overhead modify the execution engine (and other parts) of the database systems while systems that are “bolt-on” have high overhead. In this work, we demonstrate that, with the right combination of algorithmic and systems techniques, it is possible to achieve competitive performance relying exclusively on extensibility mechanisms supported by most databases. Our approach is based on a lightweight parallelism-friendly logging framework using memory-mapped files and a two-phase capture strategy that opportunistically combines logging with propagation. We present an implementation of our ideas in a system called TraceProv on top of PostgreSQL and DuckDB. TraceProv supports complex SQL queries, utilizes intra-query parallelism supported by the database system, and can handle larger-than-memory data. We experimentally compare TraceProv against SOTA relational provenance systems, demonstrating that it outperforms all existing systems.

1 INTRODUCTION

While provenance enables a wide range of use cases, adding support for provenance capture to a relational database system is a highly non-trivial task. Existing approaches fall in one of two categories: *Rewrite* systems are built on top of existing database systems and translate provenance capture computations into plain SQL code that optionally uses user-defined functions and other database extensibility mechanisms [3, 13, 33] while *native* systems extend the databases execution engine to track provenance [20, 31]. Rewrite systems are easier to port to new database systems or newer versions of an existing system, but work around limitations of SQL and of the extensibility interface of the backend database. Native systems typically modify operator implementations and have full freedom in the representation and storage of provenance and have full control over the data flow. Thus, this type of systems has achieved superior performance compared to rewrite systems. However, native approach requires significant development effort and modifications to database engine, reducing portability to new database systems and maintainability as database system code evolves.

In this work, we present TraceProv which achieves performance on par or better than native systems while being as portable as the rewrite approach. To do so, it uses a combination of algorithmic and systems techniques based on a careful analysis of bottlenecks of SOTA methods. As shown in Figure 1, TraceProv outperforms existing provenance systems on both PostgreSQL and DuckDB, including SmokedDuck [20, 22], a native system build for high performance provenance capture.

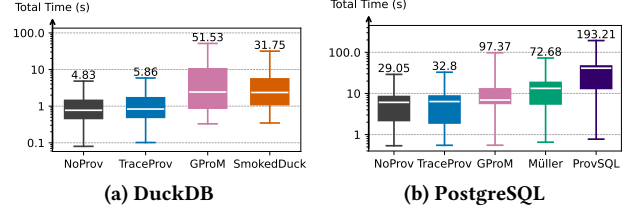


Figure 1: End-to-end time distribution for TPC-H SF10 suite on PostgreSQL & DuckDB. TraceProv is fastest on both DBMS.

EXAMPLE 1 (RUNNING EXAMPLE). Consider query Q_1 shown below which joins tables R_1 and R_2 , applies group-by aggregation to compute the count for each group $R_1.a$ and then filters out groups with less than 3 rows. Figure 2 shows example instances of the two input tables and the intermediate results of the query expressed in relational algebra (right). The join produces 5 rows belonging to two groups, out of which only one ($a = \alpha_1$) passes the **HAVING** condition. The provenance of this single result tuple according to the lineage [5, 12] provenance model (shown on the bottom left Figure 2) identifies which input tuples in R_1 and R_2 contributed to this result. These are tuples from R_1 with the group-by attribute value ($a = \alpha_1$) and their join partners in R_2 : the tuples with ids 11, 12, and 13 in R_1 and 21, 22, and 23 in R_2 .

```
Q1: SELECT R1.a, count(*)
      FROM R1 JOIN R2 ON R1.id = R2.b
      GROUP BY R1.a
      HAVING count(*) > 2;
```

To capture lineage for an SQL query, it is necessary to track data dependencies between output and input tuples of the operators in the query. The current SOTA systems primarily differ in their strategy for tracking such dependencies, namely: Annotation propagation vs. logging.

Annotation propagation vs. logging. *Annotation propagation* systems annotate input rows with unique identifiers and propagate and combine these annotations during query processing such that each (intermediate) result is annotated with its provenance. This makes annotation theoretically effective at retrieving provenance for a result tuple since it is already inlined. However, such annotations are typically variable sized since relationship between input and output tuples of a query operator can be data-dependent, and thus need non-1NF representation [30] or the result rows are duplicated to distribute provenance across copies [3, 13]. This increases the intermediate result size (either the number of rows or the size of individual rows) compared to vanilla execution, and significantly degrades performance. The intermediate result size blowup occurs for operators where single result row depends on multiple input

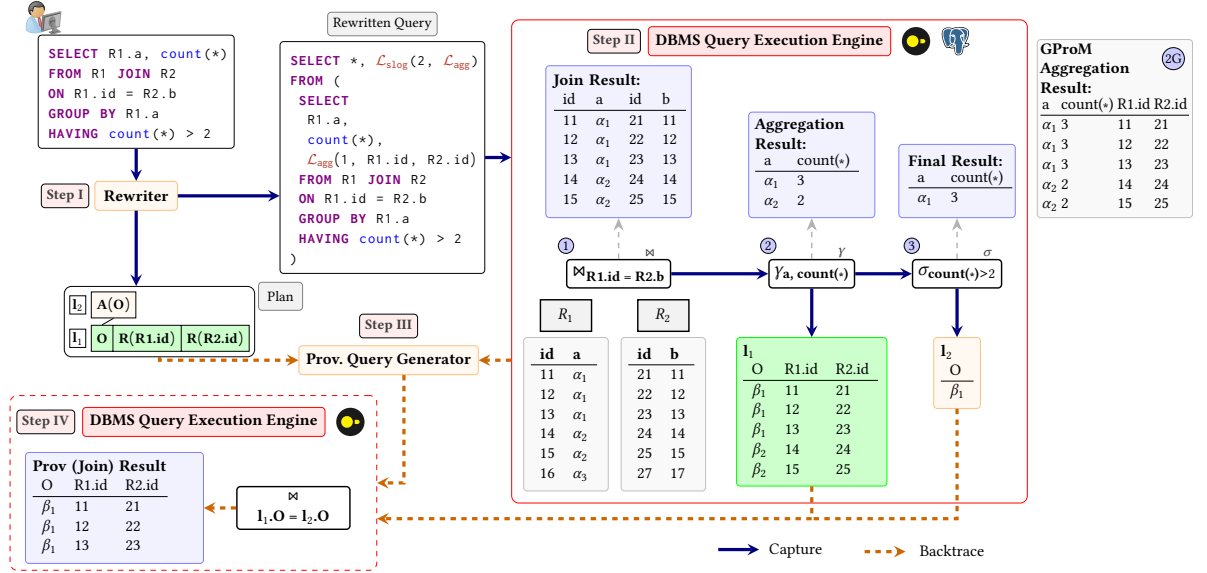


Figure 2: Capturing lineage for an example aggregation query ($Q_1 : \sigma(\gamma(R_1 \bowtie R_2))$) with TraceProv. (I) instrumenting the query to log (using UDFs (red)) and propagate provenance, and store relation between logs in plan; (II) execute the instrumented query which causes provenance to be logged in two logs backed by mmaped file (or memory only); (III) generate an SQL query that joins these logs; (IV) execute this query to produce the lineage of Q_1 (bottom left).

rows (e.g., aggregation), and when the provenance of a row is significantly shared, such as (e.g., an uncorrelated nested subquery’s provenance is shared across all result tuples produced by the outer query). In the running example in Figure 2, the intermediate result size after ② consists of two rows. However, GProM will emit 5 rows for the same aggregate (shown in ②C). Thus, successive operators, e.g., ③, will consume more rows, increasing latency.

The alternative to annotation propagation is to *log* sufficient information during query execution and then reconstruct provenance information from these logs. In the running example, for the same aggregate ②, TraceProv outputs the original number of rows (2). TraceProv does so by storing the join-result to group mapping in I_1 . Here, provenance of output tuple of the aggregate can be extracted from I_1 . Thus, creating logs avoids the intermediate result size blowup, at the cost of logging resulting in a tradeoff between annotation propagation and logging. The former is cheaper, while the latter avoids intermediate result size blowup. In general, TraceProv utilizes both, annotation propagation and logging. TraceProv performs logging at operators where many input rows may contribute to an output row (e.g., aggregation, nested subqueries) via UDF and UDA. Existing systems such as Muller and ProvSQL for PostgreSQL create similar logs via UDA and UDF in database tables and mmaped file respectively. Notably, SmokedDuck [20] modifies the execution engine of DuckDB to log mappings between output and input rows of physical operators as columnar in-memory data structures¹.

Instrumenting Queries for Capture. In Step I in Figure 2, the input SQL query is instrumented to propagate and log provenance

¹SmokedDuck does actually log at the granularity of primitive columnar operations (“function-level”) used within the implementation of a physical operator in DuckDB.

information. At this step, it also generates a plan which stores the schema of the logs and the how the logs should be joined to obtain provenance of the query. TraceProv performs logging via UDA and UDF. In Figure 2, these correspond to L_{agg} and L_{slog} . In contrast to systems like GProM which may significantly alter the structure of the input query leading to inefficient query plans [26, 27], TraceProv in most cases preserves the original query structure. Importantly, TraceProv does not alter the execution engine of the database system and, thus, has high portability. In fact, porting initial (unoptimized) TraceProv from PostgreSQL to DuckDB took less than a week of implementation.

Efficient, parallel, and append-only logging. To achieve competitive logging performance to SmokedDuck, TraceProv provides an efficient logging infrastructure for logs backed by mmaped file files or in-memory pages. We strictly perform append-only logging and memory accesses during logging. The logical log can be partitioned into multiple physical logs, allowing parallel workers (threads of processes) to write into same logical log without blocking. We further maintain zonemaps to optimize reconstruction of provenance from logs. Muller requires each log entry to be distinct, leading to expensive inserts. ProvSQL requires interprocess communication to a background writer process for log inserts. Further, both systems do not support concurrent inserts. Indeed, SmokedDuck is the only other system that does so.

Efficient Backtrace. After the logging, the logs have to be joined to backtrace the provenance one step at a time. This can be done for all result rows or an individual query result row the user is interested in. TraceProv by default computes lineage, but also supports provenance polynomials for SPJU queries. Based on the plan, TraceProv constructs a SQL query (Step III) that is then executed

using DuckDB irrespective of the DBMS for which provenance is captured (Step IV). Logs are accessed using custom TableFunctions.² For our running example, the constructed query for reconstructing lineage is a join: $I_2 \bowtie I_1$. Muller enforces bottom-up reconstruction by evaluating the input query a second time over sets of row identifiers as provenance. This is inefficient as lineage is generated for every intermediate result even if it does not contribute to final result. SmokedDuck avoids this issue by reconstructing provenance through backtracing starting from the final result. This system constructs in-memory index structures (using C++ STL data structures) for efficiently serving multiple provenance requests for the same query. While index usage reduces backtrace time, the index building cost is too high to amortize in most scenarios. TraceProv instead builds only lightweight data structures for partitioning pruning and data skipping with low logging overhead.

Contributions. In this work, we present traceprov, a new provenance system that is competitive with native systems like SmokedDuck [20, 21] and has the portability and bolt-on characteristics of approaches like ProvSQL [32, 33], GProM [3], DataProv [30], and Muller [8, 24]. This is achieved by leveraging DBMS extensibility to provide a lightweight logging framework which captures sufficient information during query execution and then subsequently using logged data to derive lineage. Achieving this trade-off is highly non-trivial and required a principled combination of existing and novel techniques.

This work makes the following contributions:

- We present a light-weight, append-only, parallel logging framework for provenance using mmap (in-memory or backed by files) to store mappings between operator input and output tuples. It tracks sufficient statistics for partition pruning and data skipping during backtrace.
- DBMS-specific wrappers for this framework are used to instrument queries for writing logs and propagating row identifiers according to a plan our system produces, preserving in most cases the structure of the original query. The logging framework is accessed through UDFs and UDAs.
- Provenance is reconstructed in backtrace from logs using SQL queries and foreign data wrappers to access the logs, exploiting DuckDB’s execution engine to reconstruct provenance top-down, utilizing partition pruning and data skipping.
- We present an extensive experimental evaluation of TraceProv on DuckDB and PostgreSQL, comparing against SOTA provenance systems for both DBMS. Overall, TraceProv outperforms all other systems. The system scales well in both data size (we present results for TPC-H up to SF 100) and degree of available parallelism (up to 10 threads).

2 RELATED WORK

Significant work has studied model for fine-grained provenance for relational queries. *Lineage* [5], one of the first proposed models, determines for each result tuple $t \in Q(D)$ of a query Q a subset of the input database D that is sufficient for producing t . This

type of provenance is sufficient for reproducing query results and tracking erroneous outputs back to the input tuples responsible for these outputs. For some use cases, we are not interested in what inputs contributed to individual output tuples, but just need a subset $D' \subseteq D$ of the input database that is sufficient for regenerating the query’s output: $Q(D') = Q(D)$. Such a subset is often referred to as a *witness* [? ? ?]. As computing a witness (and, thus, also lineage) of minimal size is computationally hard [?], TraceProv, like other relational provenance systems, does not guarantee that the lineage or witnesses it produces are minimal. Important use cases of provenance including probabilistic query evaluation [34], what-if queries [7, 22] and view maintenance [29], and how-to queries and deletion propagation [4, 17, 19] require knowledge about how input tuples were combined to generate a result. The semiring annotation model [16] models such information as annotations on data. Using the semiring of provenance polynomials, tuples are annotated with symbolic expressions that encode how input tuples (represented as variables in these expressions) have been combined (addition represents disjunctive use while multiplication represents conjunctive use) to compute a result. For example, if a query result tuple is annotated with a polynomial (symbolic expression) $x^2 + x \cdot y$ then this encodes that there are two alternative ways to derive this result: (i) by joining the input tuple x with itself or (ii) joining input tuple x with y . Extensions beyond positive relational algebra [2, 6, 10, 11] may additionally replace attribute values with symbolic expression, e.g., to record how an aggregation function result has been derived.

Systems that add support for provenance capture to relational database differ in the provenance models they support, how they represent provenance information internally, how provenance capture (recording provenance for a query) is implemented, and what kind of queries over captured provenance information are supported.

Trio [1, 23] was one of the first systems for capturing lineage and was implemented as a python-based frontend to PostgreSQL that instruments SQL queries to call a UDF that internally executes queries to capture provenance. Perm [13–15] is a modified PostgreSQL system that compiles SQL with provenance requests into plain SQL. GProM [3] implements the same approach in a middleware that supports multiple database backends. Both systems use a flat representation of provenance polynomials for positive relational algebra (and lineage for queries involving other language constructs such as aggregation and nested subqueries) where each output row maps one result tuple of the input query to tuples in its provenance. However, the system also supports producing a string representation of provenance polynomials. DataProv [30] rewrites queries to produce provenance polynomials encoded as strings. ProvSQL [33] is a postgres extension that enables capturing of semiring provenance. The system uses a new background worker process to write provenance polynomials as factorized arithmetic circuits to memory mapped files. Queries are instrumented by adding calls to UDFs that communicate with the background worker to create circuit gates.

Smoke [31] is a native approach that captures lineage by piggybacking on existing data structures generated during query execution to capture mappings between input and output tuples of

²A TableFunction in DuckDB serves the purpose of a *foreign data wrapper* in SQL/MED, i.e., it exposes external data sources as tables and optionally supports evaluating filter conditions before rows are returned to DuckDB.

individual operators in a query. To determine which inputs an output tuple of a query was derived from, these mappings for individual operators have to be joined in a second phase, effectively reconstructing lineage top-down. SmokedDuck [20, 21] is a newer native that extends DuckDB. Like Smoke, SmokedDuck does log input-output tuple mappings for individual operators in a query.

Muller [8, 24, 25] also captures lineage in two phases. However, in contrast to SmokedDuck, the system supports PostgreSQL and is based on UDFs that write mappings to tables in the database. The system ensures that provenance is only logged once, but this comes at the cost of having to run a query to check existence of a mapping in a logging table for every logged tuple mapping. The second phase of the system works bottom-up, which can be detrimental for queries that produce intermediate result with large provenance that is filtered out eventually.

3 OVERVIEW

TraceProv currently supports provenance capture for PostgreSQL and DuckDB. The system is built using extensibility mechanisms supported by the database backend, and leverages a database-independent logging framework. Thus, porting the approach to new databases or new versions of an existing database is low effort. For instance, adding support for an older version of DuckDB for experiments against SmokedDuck required only 10 hours of work, pending optimizations. We support queries composed of arbitrarily nested and correlated SPJA blocks, along with window functions and set operations. The system computes the provenance of a query in two phases. The logging is executed if a user requests the system to capture provenance for a query Q . The system, first, rewrites Q to its instrumented version, which writes provenance information to logs through calls to strategically inserted UDFs and UDAs that wrap the system's logging framework. With a few exceptions, TraceProv preserves the structure of the input query. Thus, unlike systems like GProM which generate significantly more complicated capture queries that are hard to optimize, the instrumented query generated by TraceProv typically has the same execution plan as the input query. During rewriting, we also construct a plan which encodes the schema of the logs generated for the query, and how they are related to each other, e.g., which provenance attributes they share. The instrumented query is then executed using the database backend, causing the provenance of Q to be written to the logs stored in memory-mapped files or memory regions not backed by a file. We allow multiple threads / processes to concurrently write into the same logical log without synchronization via thread/process specific physical logs. Hence, we do not impede the execution engine's parallelism.

After the logging has finished for a query Q , the user can submit requests to retrieve the provenance of Q . We refer to processing such a request as *backtrace*. TraceProv supports four types of requests: (i) return a *witness*, i.e., a subset of the tuples from the input dataset that is sufficient for producing the same result as Q . For this, TraceProv constructs one query per input relation R_i of the query that returns the row identifiers of tuples in R_i that belong to the witness. Note that a separate query is constructed for each table access, even if the same table is accessed multiple times; (ii) the second type of request returns the lineage of each individual

query result tuple of Q modeled as mappings (binary table) between output tuple ids and input tuple ids. For this case, the queries we construct return these mappings; (iii) the third type of request returns the lineage of a single query result tuple identified through its tuple identifier; (iv) the last type of provenance request computes the provenance polynomial for each **query result stored in a binary table mapping result tuple ids to polynomials**. For all types of provenance requests, the queries trace provenance top-down from the final query result back to one of the input tables by joining the logs written during the logging phase. These queries are executed in DuckDB using table-valued UDFs for reading from the captured logs. The plan generated in the first phase guides the system what logs were created and how they have to be joined. TraceProv can partition logs and automatically builds zonemaps for logs that are utilized by backtrace queries. For example, if user asks for the lineage of specific query result, we apply partition pruning to skip partitions that are guaranteed to not contribute to provenance of that result. TraceProv also supports parallel reading of logs which allows backtrace queries to fully leverage the parallelism offered by DuckDB.

4 CAPTURE INSTRUMENTATION

In this section, we discuss how TraceProv instruments a query to log provenance information in the logging phase. The instrumentation uses a set of rewrite rules to inject propagation of provenance data and logging of propagated provenance into the input query. We present these rewrites in relational algebra.

4.1 Relational Algebra

We use relational algebra with bag semantics. We use common notation for operators: projection (Π), selection (σ), join ($\bowtie$), left outer join ($\ltimes$), set union ($\cup$), bag union ($\uplus$), set intersection ($\cap$), and bag intersection ($\cap_b$). We allow projection to project onto a list of scalar expressions and apply renaming, e.g., $\Pi_{a+b \rightarrow c}(R)$ returns a single attribute c whose values are computed by adding attributes a and b . We denote aggregation grouping on attributes G and applying aggregation function $\mathcal{F}$ to input attribute a as $\gamma_{G;\mathcal{F}(a)}$. For aggregation without group-by we drop G . We also allow multiple aggregation functions to be used in a single aggregation. The duplicate elimination operator δ is denoted by δ . To model windowed aggregation supported in SQL, we introduce a window operator $\omega_{\mathcal{F}(a);G;O}^{[l,u]}(R)$ which extends each input tuple t from R with an additional attribute storing the result of computing aggregation function $\mathcal{F}$ on attribute a for a window (subset of R) determined as follows. We partition R on the partition-by attributes G and select the partition containing tuples t' where $t'.G = t.G$. Tuples within each partition are then ordered on attributes O , breaking ties arbitrarily. Finally, all tuples within $[l, u]$ of the position of t in the sort order are included in the window for t . For instance, **SELECT sum(a) OVER (PARTITION BY b ORDER BY c ROWS BETWEEN 3 PRECEDING AND CURRENT ROW) FROM R** can be expressed as $\omega_{\text{sum}(a);b;c}^{[-3,0]}(R)$.

We allow nested subqueries inside selection and projection expressions. As in SQL, subqueries can reference attributes from the schema of the operator they are nested within. For a given input tuple t from an operator whose parameters contain a nested

subquery, we substitute correlated attribute references within the nested subquery with values from t and then evaluate the subquery. We support the following types of nesting: scalar subqueries Q which evaluate to a single value, EXISTS subqueries $\Phi_{\exists}(Q)$ which evaluate to true if Q returns a non-empty result, and ANY (ALL) subqueries $\Phi[\exists t : \vec{a} \text{ op } t](Q)$ ($\Phi[\forall t : \vec{a} \text{ op } t](Q)$) which evaluate to true if comparison $a \text{ op } t$ evaluates to true for at least one (all) tuples t returned by Q . As an example, consider the SQL query shown below and the corresponding query in relational algebra.

```
SELECT a
FROM R
WHERE EXISTS (SELECT *
              FROM S
              WHERE R.b = S.c)
OR a < 5;
```

$$\Pi_a(\sigma_{\Phi_{\exists}(Q_1) \vee a < 5}(R))$$

$$Q_1 = \sigma_{b=c}(S)$$

4.2 Capture Plans and Log Functions

As mentioned previously, TraceProv's rewrite combines propagation with logging of provenance attributes of a query instead of always propagating (like GProM) or always logging (like Smoked-Duck). when instrumenting a query Q for provenance capture, TraceProv constructs a *plan* PLAN_Q that records at which operators of the query a log $\mathbf{l}$ will be written and what provenance attributes will be logged. Formally, a plan PLAN_Q is a set of logs $\{\mathbf{l}_i\}$ where each log is a tuple

$$\mathbf{l}_i = (\mathcal{E}, P, p)$$

where i is a unique identifier for the log within the scope of the query. Pare the provenance attributes of the log, i.e., the schema of the records that are written to the log. $\mathcal{E}$ is a list of entries which each stores metadata about a single or group of provenance attributes that are logged at $\mathbf{l}_i$. In addition to storing which provenance attributes belong to the entry, an entry records where these attributes originated from (e.g., an input relation or another log), and their type (e.g., produced by an aggregation). As an example, consider the plan for our running example shown in Figure 2. $\mathbf{l}_1$ contains entries O, R(R1.id), and R(R2.id). The first entry (O)'s type encodes that the logged attribute is a tuple identifier for intermediate results produced at the operator (in this case generated by TraceProv's log function $\mathcal{L}_{\text{agg}}$ called at the instrumented aggregation operator in the query). The latter two entries, R(R1.id), and R(R2.id), are base relation entries (type R) which store what attributes from a particular input relation are used as provenance attributes to identify tuples from this input relation (R1.id and R2.id in the example). For $\mathbf{l}_1$, the schema of the log has one provenance attribute per entry, but in general one entry may correspond to multiple attributes, e.g., if a table has a multi-attribute key that we use to identify tuples from this table. $\mathbf{l}_2$ contains only 1 entry, A(O) with type A. This type signifies that the entry's provenance attribute was produced by a prior $\mathcal{L}_{\text{agg}}$ call, in this case, line 6 of the rewritten query. Furthermore, the edge between $\mathbf{l}_2$.A(O) and $\mathbf{l}_1$.O, in this case, is interpreted as a sign to join the logs $\mathbf{l}_1$ and $\mathbf{l}_2$ using the join condition $\mathbf{l}_2$.A(O)= $\mathbf{l}_1$.O during the backtrace

As evident, the plan serves three important roles. First, it records entries that encode the schema of the log. Second, it stores how the log files are interconnected through entries. Third, the type of entry, e.g., A determines *how* any of its edges with any other

entry in a different log should be interpreted in backtrace to derive lineage. We illustrate later how types other than A interpret edges between entries differently to derive lineage query.

Accordingly, the goal of the rewriter is two fold: 1. instrument the query and interject the TraceProv logging aggregates and functions, and 2. construct the plan that contains the appropriate entries for a given log and contains its connections to other logs that are utilized during backtrace.

Before we describe how the rewrite system achieves the above goals, we introduce additional notation for TraceProv user-defined functions and aggregates, and notations to understand the rewrite rules.

4.2.1 Logging functions. TraceProv uses two logging functions which are $\mathcal{L}_{\text{slog}}(\mathbf{l}_i, c_1, \dots, c_n)$ and $\mathcal{L}_{\text{agg}}(\mathbf{l}_i, c_1, \dots, c_n)$. Here, $\mathbf{l}_i$ is a log to which to append to and $c_1, \dots, c_n$ are the values to be written to the log which are propagated provenance attributes (tuple identifiers), tuple identifiers for intermediate results, or $\mathcal{L}_{\text{slog}}$ writes a log record and returns true independent of its input. Function $\mathcal{L}_{\text{agg}}$ is a UDA that returns a 64-bit tuple identifier for an output tuple produced by aggregation and stores a mapping between provenance attributes from the input of the aggregation and the generated tuple identifier.

4.2.2 Provenance Attributes and Entries. Each rewrite rule for a given query Q produces 4 items: $(Q^+, \mathcal{P}, \mathcal{E}, \mathbf{l})$. Here, Q^+ is the instrumented query with user-defined functions and aggregates. $\mathcal{P}$ refer to the *provenance attributes* of query Q , i.e., any provenance attributes that were simply propagated. $\mathcal{E}$ is the *provenance entries* or simply entries corresponding to the *kind* of provenance attributes ($\mathcal{P}$). Finally, $\mathbf{l}$ consists of the *provenance entries* that got logged. For example, for a simple projection, the rewrite system may decide to simply propagate provenance attributes and entries. In this case, while $\mathcal{P}$ and $\mathcal{E}$ will be set, $\mathbf{l}$ will be empty. However, in aggregation, as shown later in the rewrite rules, $\mathbf{l}$ will contain the O and the provenance entries of the child relational expression.

4.2.3 Provenance Entry Types. Each entry in $\mathcal{E}$ has a specific *entry type*, e.g., R or A. Some *entry types*, e.g., A, require an edge to another entry which introduces the interconnection between log entries, and correspondingly, logs.

4.3 Capture Rewrite Rules

TraceProv instruments queries for provenance capture using a set of rewrite rules. For a query Q we use Q^+ to denote the rewritten query after instrumentation using the rules. The most important rules are shown in Figure 3. The remaining rules are shown in [18]. In addition to returning Q^+ , each rule computes the provenance attributes $\mathcal{P}$ propagated for the rewritten query, its log entries $\mathcal{E}$, and, optionally, creates a new log $\mathbf{l}$ for the plan for the query. The final plan PLAN_Q is then simply the set of generated logs. We associate each operator in a query with a unique identifier with ID_Q denoting the identifier for the root operator of an algebra expression Q . We use $\mathbf{R}$ to denote the schema of relation R and employ the same notation to denote the schema of queries.

Consider first the rule **R1** for a table access operator. This operator is rewritten by adding a projection that propagates the primary key attributes $\text{PK}(R)$ of relation R as provenance attributes using

renaming to create unique names for each attribute as follows: an attribute $a \in \text{PK}(R)$ is renamed as $\mathcal{T}_{aR}$. Note that in the rewrite rule $\overrightarrow{\mathcal{T}_{\text{ID}(R)}}$ denotes the list of these new attribute names, and $\parallel$ denotes concatenation of lists. No log is written at a table access operator.

A selection (**R4**) is rewritten by applying the selection to its rewritten input. However, selection conditions may contain nested subqueries that have to be rewritten. This is done by function rewrExpr which we will cover in Section 4.4. The provenance attributes (log entries) of a selection are the same as the provenance attributes (log entries) of its input.

For projection (**R5**), we append the provenance attributes of the input to the projection expressions. Recall that projection expressions are allowed to contain nested subqueries. Thus, like for selection the expression (and the contained subqueries) have to be rewritten.

Joins are rewritten (**R2**) by joining the rewritten inputs of the join. The provenance attributes for the join is the concatenation of the provenance attributes of both inputs. The same applies to the log entries. No log is created for joins.

For bag union (**R6**), TraceProv computes the union of the rewritten inputs. However, as the two inputs may differ in their number of provenance attributes, we have to pad the schema of the input with less provenance attribute with null values if necessary. Furthermore, in the backtrace phase we need to be able to determine which of the two input an output was derived from. In the rewrite rule we use $\mathcal{NE}(Q_i, Q_j)$ to denote a list of $\max(0, |Q_i| - |Q_j|)$ null values used for padding. Furthermore, $\mathcal{P}_{Q_1, Q_2}^{\max}$ returns the provenance attributes of either Q_1 or Q_2 whichever one is longer. For that we add a new attribute that stores the identifier of the operator. The log entry for bag union records the name of the attribute identifying the input a tuple stems from as well as the lists of entries for both inputs of union. This information is used during backtrace to understand which attributes are derived from which inputs of the union. Note that we do not write a separate log for union.

For aggregation (**R3**), an output tuple for a group depends on all input tuples from that group. As mentioned earlier, TraceProv does create new tuple identifiers for aggregation outputs and logs records that store the mapping between these new tuple identifiers and provenance attributes from input tuples of the aggregation. In the rule we use $\mathcal{F}$ to denote a list of aggregation functions. The aggregation is rewritten by adding a call to log function $\mathcal{L}_{\text{agg}}$ which writes these mapping records to the new log created for the aggregation. A new name $\mathcal{T}_{\text{ID}(Q)}$ is used for the attribute storing the tuple identifiers generated by $\mathcal{L}_{\text{agg}}$. The log entry for aggregation $A(\mathbf{I}, \mathcal{T}_{\text{ID}(Q)})$ stores the log $\mathbf{I}$ created for the aggregation and the name of the tuple identifier attribute.

The final instrumented query Q_{log} used by TraceProv to capture provenance for Q may require an additional final logging step unless the rewrite rules create a log for the root operator of the query below. We use $\mathbf{I}(Q)$ to denote the log, if any, written by the root operator of query Q . Q_{log} is defined as shown below.

$$Q_{\text{log}} = \begin{cases} Q^+ & \text{if } \mathbf{I}(Q^+) \neq \emptyset \\ \Pi_{Q^+, \mathcal{L}_{\text{slog}}(\mathbf{I}_{\text{ID}(Q)}, \mathcal{P}(Q))}(Q^+) & \text{otherwise} \end{cases}$$

EXAMPLE 2 (QUERY INSTRUMENTATION). Figure 4 shows the relational algebra tree for the running example query, the query instrumented for provenance capture, and plan. The two table access operators were rewritten by adding a provenance to duplicate and rename their primary key attributes. These attributes are written to log $\mathbf{I}_2$ at the rewritten aggregation using log function $\mathcal{L}_{\text{agg}}$. Then finally as the root of the rewritten query (selection) does not have an associated log, another projection is added in Q_{log} to log which aggregation results did fulfill the selection condition.

4.4 Rewriting Expressions and Subqueries

Recall that we allow condition selections and projection expressions to contain nested subqueries which may reference attributes from the schema of the operator in which the subquery occurs. The subquery will be evaluated for each tuple from the outer operator. When logging provenance for a tuple t_{inner} produced by one such evaluation of the nested subquery, we need to record for which tuple t_{outer} from the outer query this tuple was produced such that during backtrace the provenance of t_{outer} can be traced back to the provenance of t_{inner} . TraceProv achieves this by adding additional correlations with provenance attributes from the outer query that are included in the top level log written inside the nested subquery.

Let $\Phi(Q)$ denote a subquery of one of the supported types, e.g., $\Phi_{\exists}(Q)$ or $\Phi[\exists t : a \text{ op } t](Q)$. For a scalar expression e , we use $\text{subqueries}(e)$ to denote the subqueries that occur in e and $e[e_1 \leftarrow e_2]$ to denote replacing all occurrences of e_1 in expression e with e_2 . Recall that within the scope of a query we identify each operator with a unique identifier. Thus, substituting a nested subquery in an expression replaces only this specific instance of the nested subquery even if the same subquery expression appears more than once in the expression. Algorithm 1 shows the pseudocode for function $\text{rewrExpr}(e, \mathcal{P})$ called from the algebraic rewrite rules to rewrite projection expressions and selection conditions. The function takes as input an expression e to rewrite and the provenance attributes $\mathcal{P}$ from the outer query. It replaced every subquery $\Phi(Q)$ in e with a rewritten version which is generated by rewriting Q and then uses $\mathcal{L}_{\text{vlog}}$ to write the provenance attributes from the outer query and the provenance attributes of Q to a new log.

It is sometimes possible for TraceProv to determine that an expression evaluates to a fixed result irrespective of the result of a nested subquery. For instance, in $\sigma_{a=3 \vee \Phi_{\exists}(Q)}(R)$ for any tuple t from R with $t.a = 3$ the selection condition evaluates to true independent of whether $\Phi_{\exists}(Q)$ returns true or false. If we can detect such situations efficiently, then we can avoid logging provenance for this specific execution of the nested subquery to reduce the size of the captured lineage / witness. Consider a selection condition θ containing a nested subquery $\Phi(Q)$. To apply this optimization, TraceProv transforms the selection conditions θ into disjunctive normal form and creates a guard θ_{guard} . If the guard evaluates to false then θ 's result is independent of the result of the nested subquery and, thus, the provenance for this execution of the nested subquery does not have to be captured. The guard is then injected into the rewritten nested subquery as follows to avoid writing to the subquery's top-level log if the guard fails:

$$\Pi_Q(\Pi_Q \parallel \mathcal{L}_{\text{vlog}}(\mathbf{I}_{\text{ID}(Q)}, \mathcal{P} \parallel \mathcal{P}(Q))(\sigma_{\theta_{\text{guard}}}(Q^+)))$$

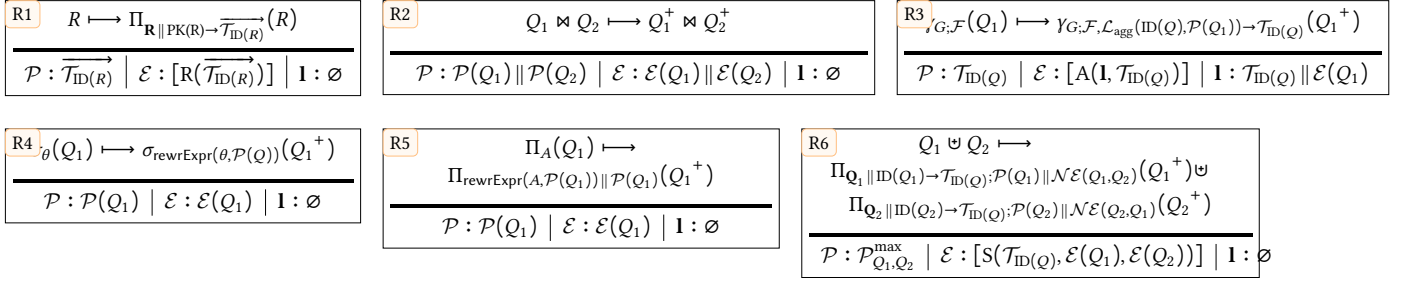


Figure 3: Rewrite rules for instrumenting queries for provenance capture

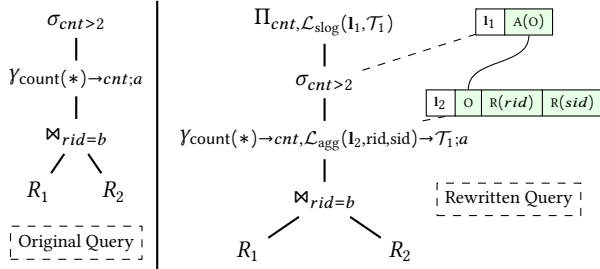


Figure 4: Plan and instrumented query for the running example query.

Algorithm 1 $\text{rewrExpr}(e, \mathcal{P})$

```

for  $\Phi(Q) \in \text{subqueries}(e)$  do
   $\Phi(Q)^+ \leftarrow \Pi_Q(\Pi_Q \parallel \mathcal{L}_{\text{vlog}}(\text{ID}(Q), \mathcal{P} \parallel \mathcal{P}(Q)))(Q^+)$ 
   $e \leftarrow e[\Phi(Q) \leftarrow \Phi(Q)^+]$ 
return  $e$ 

```

5 BACKTRACE

Backtrace is the second phase of provenance capture with TraceProv which returns the lineage of the input query Q using one or more queries $\mathbb{Q}_{BT}$ that each trace back from the result of the input query Q to one of the input relations accessed by Q by joining logs generated by the instrumented query Q_{log} in the first phase. Recall that as a side effect of constructing Q_{log} that generates the logs, we construct a plan PLAN_Q which records which logs are written during logging, their schemas, and additional metadata such as for a given log from which other logs or base relations its inputs originate from. Thus, PLAN_Q determines how the logs have to be joined to trace from query output tuples back to contributing tuples from a particular input relation one step at a time. The queries in $\mathbb{Q}_{BT}$ are always executed using DuckDB. For PostgreSQL, the result of this query is exposed through a table access method that evaluates queries from $\mathbb{Q}_{BT}$ using DuckDB incrementally.

Recall that logs are stored in a custom binary format in mmaped file areas in memory, optionally backed by files. To access logs from within a backtrace query $\mathbb{Q}_{BT}$, we provide a table-valued function $\mathcal{R}$ for DuckDB which reads a log $\mathbf{I}_i$ and returns it as a DuckDB table. We refer to this function as the *log reader*. Next we first describe the implementation of the log reader $\mathcal{R}$ and then present how the SQL queries in $\mathbb{Q}_{BT}$ are constructed based on PLAN_Q .

TraceProv supports four types of provenance requests for a query Q : (i) return a *witness*, i.e., a subset of the tuples from the input dataset that is sufficient for producing the same result as Q . For this, TraceProv constructs one query per input relation R_i (leaf node of the query's algebra tree) that returns the row identifiers of tuples in R_i that belong to the witness; (ii) the second type of request returns the lineage of each individual query result tuple modeled as mappings (binary table) between output tuple ids and input tuple ids. Each query in $\mathbb{Q}_{BT}$ returns one such mapping; (iii) the third type of request returns the lineage of a single query result tuple identified through its tuple identifier; (iv) the last type of provenance request computes the provenance polynomial for each query result stored in a binary table mapping result tuple ids to polynomials.

5.1 The Log Reader

The log reader is implemented as a DuckDB table function $\mathcal{R}(\mathbf{I}_i, \text{offset})$ that takes as input a log $\mathbf{I}_i$ it should read from and optionally an offset. The optional offset is used when computing the lineage for a single result tuple (as we will expand on further in Section 5.2). Each logical log may be split across multiple physical partitions (distinct areas in memory or separate files on disk) for concurrent inserts. DuckDB allows parallel reads from table functions. We currently assign each DuckDB thread to a partition of $\mathbf{I}_i$, and thus each partition is read concurrently. We use lightweight statistics generated during logging to expose table and column level statistics which significantly aid backtrace query optimization. We also allow filter pushdown for prefiltering by zonemaps inserted during logging, i.e., skipping regions of partition during scanning that cannot satisfy filter condition. To perform partition pruning, we check if the pushed down filters can be satisfied only by a subset of partitions using partition-level statistics created during logging. If parameter offset is provided, then only the row at this position will be returned. Note that in TraceProv the top-level log always contains a single row per result tuple. Thus, retrieving the entry for the i^{th} result tuple corresponds to reading the i^{th} row in the log.

5.2 Backtrace Query Construction

We will first describe the backtrace query construction when computing the lineage for every query result tuple as binary tables and then discuss how other request types are handled. The main entry point is backtrace which takes a plan PLAN_Q for a query Q as input and returns $\mathbb{Q}_{BT}$ which associates each table access R in

Algorithm 2 $\text{backtrace}(\text{PLAN}_Q)$

```

1  $\mathbb{Q}_{BT} \leftarrow \emptyset$ 
2  $\mathbf{l}_{root} \leftarrow \text{ROOT}(\text{PLAN}_Q)$  // Get top-level log
3  $Q_{root} \leftarrow \mathcal{R}(\mathbf{l}_{root})$  // Read top-level log
4  $\text{backtraceEntries}(Q_{root}, \mathbf{l}_{root}, \mathbb{Q}_{BT})$ 
5 return  $\mathbb{Q}_{BT}$ 

```

Algorithm 3 $\text{backtraceEntries}(Q_{cur}, \mathbf{l}_{cur}, \mathbb{Q}_{BT})$

```

1 for  $\mathbf{l}_{sub} \in \text{subQ}(\mathbf{l}_{cur})$  do
2    $\text{backtraceSub}(Q_{cur}, \mathbf{l}_{sub}, \mathbb{Q}_{BT})$ 
3 for  $L \in \mathbf{l}_{cur}$  do
4   if  $L = R(P_1, \dots, P_n)$  then
5     if  $\mathbb{Q}_{BT}[R] = \emptyset$  then
6        $\mathbb{Q}_{BT}[R] \leftarrow \Pi_{O, P_1, \dots, P_n}(Q_{cur})$ 
7     else
8        $\mathbb{Q}_{BT}[R] \leftarrow \mathbb{Q}_{BT}[R] \cup \Pi_{O, P_1, \dots, P_n}(Q_{cur})$ 
9   else if  $L = A(\mathbf{l}_{child})$  then
10     $\text{backtraceAgg}(L, Q_{cur}, \mathbf{l}_{child}, \mathbb{Q}_{BT})$ 
11   else if  $L = W(\mathbf{l}_{child})$  then
12     $\text{backtraceWin}(L, Q_{cur}, \mathbf{l}_{child}, \mathbb{Q}_{BT})$ 
13   else if  $L = S(s_{id}, [\mathbf{l}_1, \dots, \mathbf{l}_k])$  then
14     $\text{backtraceSet}(L, Q_{cur}, \mathbb{Q}_{BT})$ 

```

Q with a query $\mathbb{Q}_{BT}[R]$. Note that in our implementation, table accesses are by identifiers that are unique within the scope of a query. For instance, if a relation R is accessed more than once, e.g., in a self-join $R \bowtie R$, a separate query will be created for each access. backtrace initializes an empty map for $\mathbb{Q}_{BT}$ and creates an initial query Q_{cur} that reads the top-level log, i.e., the root of PLAN_Q . It then calls backtraceEntries which processes the entries of a log by dispatching their type-specific handlers e.g., entries produced by an aggregation). backtraceEntries also handles subqueries.

Recall that if a log has attributes from a subquery that contains a nested subquery (that may contain correlated references to some these attributes), then the log for this subquery is connected the outer log. backtraceEntries first iterates over all such logs connected to $\mathbf{l}_{cur}$ and uses function backtraceSub to traverse into the part of the plan for these nested subqueries. Afterwards, the algorithm processes all entries of $\mathbf{l}_{cur}$, dispatching to methods for handling specialized entries. For an entry L that corresponds an input relation, i.e., is of the form $R(P_1, \dots, P_n)$, backtrace has reached a leaf of the algebra tree. We project the current query on the output tuple identifier attribute for the query (O) and the provenance attributes $P_1, \dots, P_n$ of L . This query is then stored in $\mathbb{Q}_{BT}[R]$. If $\mathbb{Q}_{BT}[R]$ already exists, then we bag union the current query for R with the newly derived one.

Aggregate Logs. Function backtraceAgg (Algorithm 4) implements backtrace for aggregate logs. It first joins the aggregate log $\mathbf{l}_{cur}$ with the current backtracing query. If the database used a parallel implementation of aggregation where multiple workers pre-aggregate the data locally followed by a global combination (potentially parallelized) step, then TraceProv generates two logical logs for the aggregation. (1) $\mathbf{l}_{preagg}$, which is the pre-aggregation log, physically partitioned by worker such that each worker writes to its physical

log without coordination with other workers. (2) the combiner log ($\mathbf{l}_{comb}$) which stores how the pre-aggregated state across workers was combined. This log is also worker partitioned, i.e., multiple combiner workers write to their private copy without synchronization. In this parallelized aggregation case, $\mathbf{l}_{cur}$ in line 1 of Algorithm 4 refers to $\mathbf{l}_{comb}$. We also need to join with the log $\mathbf{l}_{pre}$ for the pre-aggregation step (lines 3-6). Afterwards, backtraceEntries is used to process the entries of the pre-aggregation or only log for the aggregation.

EXAMPLE 3 (BACKTRACE QUERY CONSTRUCTION). *Reconsider our running example query. Figure 4 shows the algebra tree for this query and the two logs generated during logging. $\mathbf{l}_1$ records which aggregation results pass the filter and, thus, are in the final query result while $\mathbf{l}_2$ does map groups (each group is assigned an identifier stored in O) to pairs of tuples from R_1 and R_2 belonging to this group in the aggregation's input. backtrace will generate two queries, each tracing back from the query result to one of the input relations by joining the log top-down. These queries are constructed by first initializing Q_{cur} with a log reader for $\mathbf{l}_1$ (the top-level log) in the main backtrace algorithm backtrace . Then backtraceEntries is called with this initial query and $\mathbf{l}_1$ and an empty $\mathbb{Q}_{BT}$ that will eventually hold the constructed backtrace queries. Function backtraceEntries then iterates over all entries of log $\mathbf{l}_1$. There is a single entry of type A (an aggregation result log). This entry is processed by backtraceAgg by joining the current query (reading log $\mathbf{l}_1$) with the log referenced in the entry ($\mathbf{l}_2$), updating Q_{cur} to $\mathcal{R}(\mathbf{l}_1) \bowtie \mathcal{R}(\mathbf{l}_2)$. Then backtraceEntries is called again to handle the entries of $\mathbf{l}_2$. When processing the entry for $R(\text{rid})$, the backtrace query for R_1 is created. Similarly, when processing $R(\text{sid})$ we create the query for R_2 and store it in $\mathbb{Q}_{BT}[R_2]$. Equation 1 shows the backtrace queries for R_1 and R_2 .*

$$\begin{aligned}\mathbb{Q}_{BT}[R_1] &= \Pi_{O, \text{rid}}(\mathcal{R}(\mathbf{l}_1) \bowtie \mathcal{R}(\mathbf{l}_2)) \\ \mathbb{Q}_{BT}[R_2] &= \Pi_{O, \text{sid}}(\mathcal{R}(\mathbf{l}_1) \bowtie \mathcal{R}(\mathbf{l}_2))\end{aligned}\tag{1}$$

Algorithm 4 $\text{backtraceAgg}(Q_{cur}, \mathbf{l}_{cur}, \mathbb{Q}_{BT})$

```

1  $Q_{cur} \leftarrow Q_{cur} \bowtie \mathcal{R}(\mathbf{l}_{cur})$  // join with aggregate log
2 if  $\text{iscombined}(\mathbf{l}_{cur})$  then // Parallel agg. with combiner?
3    $\mathbf{l}_{preagg} \leftarrow \text{getPreaggLog}(\mathbf{l}_{cur})$ 
4    $Q_{cur} \leftarrow Q_{cur} \bowtie \mathcal{R}(\mathbf{l}_{preagg})$  // join with combiner log
5    $\mathbf{l}_{cur} \leftarrow \mathbf{l}_{preagg}$ 
6  $\text{backtraceEntries}(Q_{cur}, \mathbf{l}_{cur}, \mathbb{Q}_{BT})$ 

```

Algorithm 5 $\text{backtraceSub}(Q_{cur}, \mathbf{l}_{sub}, \mathbb{Q}_{BT})$

```

 $Q_{cur} \leftarrow Q_{cur} \bowtie \mathcal{R}(\mathbf{l}_{sub})$ 
 $\text{backtraceEntries}(Q_{cur}, \mathbf{l}_{sub}, \mathbb{Q}_{BT})$ 

```

Handling Nested Subqueries. Nested subqueries are handled by backtraceSub (Algorithm 5) that is passed as input the top-level log $\mathbf{l}_{sub}$ for the subquery. This log is then natural joined with the current query Q_{cur} . Note that common attributes between Q_{cur} and $\mathbf{l}_{sub}$ correspond to correlations between the nested subquery and the outer query traced by Q_{cur} . Joining on these attributes is

necessary to match the provenance of outer tuples with tuples from the corresponding evaluation of the nested subquery. Afterwards, the entries of $\mathbf{l}_{sub}$ are processed using `backtraceEntries` to continue tracing further.

Join Rewrite Optimization.

Consider a backtrace query $\mathcal{R}(\mathbf{l}_{root}) \bowtie \mathcal{R}(\mathbf{l}_1)$. If the user requests backtrace for a given offset in output result, we pass that offset to $\mathcal{R}(\mathbf{l}_{root})$ which now emit only 1 row which we store into `rootRow`. Now, the join can be converted to $\sigma_{\text{rootRow}=\vec{b}} \mathcal{R}(\mathbf{l}_1)$. Here, $\vec{b}$ are attributes from $\mathcal{R}(\mathbf{l}_1)$. In practice, only the subset of attributes used in original join are included in the predicate. We call this join rewrite optimization since join is rewritten into a selection.

6 LOG STRUCTURE AND INTERFACE

Log writer is a critical component of TraceProv, and should have minimal instrumentation overhead low, even under parallelized execution. We now discuss the several design decisions made to achieve this goal. We present the physical layout of logs and additional lightweight indexing integrated into logs for efficient retrieval of log entries during backtrace.

6.1 Log Interface

Recall that TraceProv uses UDFs, and UDAs to perform insertions into a log as a side effect of execution, and that the log locations are decided by the rewrite. The following interfaces of log writer are exposed:

- UDF $\mathcal{L}_{\text{slog}}(\mathbf{l}_i, c_1: \text{int}[], c_2: \text{int}[] \dots c_n: \text{int}[]) \rightarrow \text{True}$
- UDA $\mathcal{L}_{\text{agg}}(\mathbf{l}_i, c_1: \text{int}[], c_2: \text{int}[] \dots c_n: \text{int}[]) \rightarrow \text{int64}$

Here, $\mathbf{l}_i$ is the log location, and $c_1, c_2 \dots c_n$ are the attributes that we need to log. Each c_i is vector of `int(int32 or int64)`. The above representation allows unified handling of DuckDB and PostgreSQL. For DuckDB, UDFs and UDAs are called one chunk at a time, with each chunk containing vectors $c_1, c_2 \dots c_n$. For PostgreSQL, the row-based execution can be modelled by forcing $|c_i| = 1$.

Even though $\mathcal{L}_{\text{agg}}$ and $\mathcal{L}_{\text{slog}}$ are semantically and operationally different (aggregate vs scalar), the key observation here is that $\mathcal{L}_{\text{agg}}$'s purpose is to associate input rows to groups. To do so, $\mathcal{L}_{\text{agg}}$ simply generates a unique group identifiers (`int64`), and subsequently treats the unique identifier as an additional attribute. Thus, each row as an associated group identifier and rows belonging to the same group share the same group identifier. Hence, the additional complexity of $\mathcal{L}_{\text{agg}}$ over $\mathcal{L}_{\text{slog}}$ is now the generation of unique group identifiers. This is done by incrementing a worker- and $\mathbf{l}_i$ -specific counter each time we encounter an group without an identifier. Note that this is possible via UDAs since databases allow UDAs to identify when a group has uninitialized state. The group identifiers generated are trivially unique in a given worker.

Recall that for a given $\mathbf{l}_i$, $\mathcal{L}_{\text{agg}}$ can be executed concurrently by different workers. Since the counter is worker-local, no synchronization is needed to assign group identifiers locally. The group identifiers are made globally unique, without synchronization, by setting the 1 byte in the 8-byte representation of group identifier as the worker id. Note that worker id identifies a given thread or process in its corresponding thread/process pool. It is possible that the same group occurred across workers. In such cases, databases call a combiner to combine partial states for the same group from

different workers. We leverage this mechanism to associate which group identifiers across workers are associated to the same group. Note that combine is treated as a second level aggregation, i.e., calling $\mathcal{L}_{\text{agg}}$ again with two attributes (source, and target), thus does not extend the log representation.

6.2 Physical Log Structure

So far, we have established that both $\mathcal{L}_{\text{slog}}(\mathbf{l}_i, c_1: [], c_2: \text{int}[] \dots c_n: \text{int}[])$ and $\mathcal{L}_{\text{agg}}(\mathbf{l}_i, c_1: \text{int}[], c_2: \text{int}[] \dots c_n: \text{int}[])$ fundamentally log n and $n+1$ vectors of type `int` respectively. Recall that Section 6.1 established that $\mathcal{L}_{\text{agg}}$ effectively logs another vector for the group identifier, in addition to all the other vectors ($c_1, c_2 \dots c_n$) passed. Further, we only process only one chunk at a time.

We partition the logical log $\mathbf{l}_i$ into multiple physical logs, for each worker: $\mathbf{l}_i(\text{worker } 0), \mathbf{l}_i(\text{worker } 1), \dots, \mathbf{l}_i(\text{worker } P)$. Each worker performs logging in its corresponding physical log, without significant blocking or synchronization. A small amount of synchronization is needed to assign each worker its id from a global counter. However, it only occurs on the first log call and cached in local storage for subsequent use.

We now discuss the physical representation of the log, which needs to represent $c_1, c_2 \dots c_n$. The Figure 5 shows the physical structure of the log. Here, the layout is organized into 3 files. (i) the data file stores for each chunk, the concatenated representation of vectors $c_1, c_2 \dots c_n$. That is, for each chunk the entries are stored in a columnar fashion; (ii) the stats file stores statistics (number of entries, minimum and maximum values for each column) for each chunk in the data file; (iii) a metadata file storing global information about the physical log such as minimum and maximum column values across all chunks, the number of chunks, and pointers to the end of the data and stats files which are used when appending entries to the log.

Note that core of the logging is a simple `memcpy` operation from c_i to the current write head. Hence, logging is cheap, as we show later in Section 7.6, that overhead is in the order of nanoseconds. The storage size per log entry for c_i is either 8 bytes or 4 bytes, dictated by the maximum number of bytes needed for representation of c_i . Note that we request multiple pages (32-256) at a time from the operating system, and hence can overrequest memory. However, modern operating systems page on-demand, hence, this approach has no significant downsides.

The data file, stats file and metadata file need not correspond to actual files on the disk. That is, we allow each file to be composed entirely of in-memory pages (with no file backing). We also support huge pages (1MB) to avoid expensive page-faults.

We can further partition a worker-local log based off any column ($c_1, c_2 \dots c_n$). In practice, we partition the $\mathcal{L}_{\text{agg}}$ based off the group identifier column. This is shown in Figure 6. Here, groups with identifier β_1 is written into Bucket 1, while groups with identifiers β_2 or β_3 is written into Bucket 2.

7 EXPERIMENTAL EVALUATION

In this section, we evaluate TraceProv against current state-of-the-art system to evaluate its provenance capture performance for a variety of workloads on PostgreSQL and DuckDB, comparing against SOTA systems on both databases.

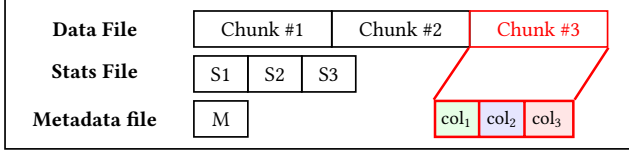


Figure 5: Physical log structure: (i) the data file stores chunks of log entries stored in a columnar fashion; (ii) the stats file records statistics for each chunk such as min/max values per column; (iii) the metadata file stores global metadata for the physical log.

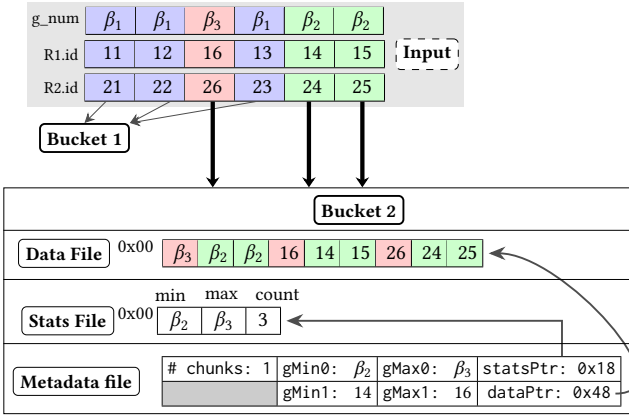


Figure 6: Physical log representation of a $\mathcal{L}_{agg}$. The log contents after the input chunk has been logged is shown. The Metadata file stores the number of chunks, and global min-max statistics of O and R1.id column

7.1 Experimental Setup & Workloads

All experiments are run on Mac Studio machine with a M2 Max 12-core CPU and 64GB of RAM. Each experiment is performed 15 times. We report median runtimes, excluding the first 5 executions (hot cache). We use PostgreSQL version 16, and DuckDB (version 1.4.3). We mainly focus on **lineage capture** as lineage is defined for a large class of relational operations including nested subqueries, aggregation, and window functions. We consider two variants: P_all computing lineage for all result tuples of a query and P_single which only requires capture for one result tuple (selected randomly unless stated otherwise). To demonstrate the broad applicability of our techniques, we also consider **provenance polynomials**: a more expressive type of provenance that encodes how input tuples were combined to derive a query result and are needed for use cases like probabilistic query processing [33, 34] and hypothetical reasoning [7, 19, 22]. TraceProv supports provenance polynomials for SPJU queries.

7.1.1 Baselines. PostgreSQL. For PostgreSQL, we compare against the rewrite-based system GProM [3], the approach proposed in [24] which we refer to as *Muller*, and *ProvSQL* [32, 33]. As mentioned in section 2, Muller [24] utilizes a two-phase approach that computes provenance bottom-up in the second phase while TraceProv backtracks starting from the query result (top-down). ProvSQL

stores provenance polynomials as circuits which are stored in memory-mapped files. ProvSQL uses a separate background worker process to log provenance to these mmaped file files. For parallel query execution, each worker process has to communicate with this background worker, limiting performance in parallel settings.

DuckDB. We also use GProM and the native lineage capture system SmokedDuck [20, 22]. **SmokedDuck** also splits lineage capture into two phases and performs the second phase top-down like TraceProv. However, in contrast to TraceProv which uses UDFs and a separate logging framework, SmokedDuck is deeply integrated into DuckDB’s execution engine, being based on DuckDB version 0.10.3. For a fair comparison, in experiments directly comparing against SmokedDuck, we run TraceProv on the same DuckDB version. Currently, SmokedDuck does not support multi-threaded query execution. SmokedDuck exists in multiple versions. We contacted the authors and based on the guidance selected *smokedduck-2025-f*. However, the second phase of this branch either crashed or returned incorrect lineage for most of the TPC-H queries. We fixed some of these bugs to be able to compare against this system.

7.1.2 Metrics. We report the execution time as measured by the database system. For PostgreSQL, we use “Execution Time” as reported by running the query with **EXPLAIN ANALYZE** (timing off). For DuckDB, we report latency (or timing for SmokedDuck) after turning on query profiling. In some experiments, we also report storage size of the logs written by a system.

7.2 TPC-H Overview

TPC-H SF10. Figure 7 shows the runtime of lineage capture for TraceProv and all baselines on TPC-H SF10 for lineage capture for a single row (P_single) and for all result tuples (P_all). In the plots, we group queries by their structure. For each query we also report the runtime of the query without provenance capture as a reference point (noprov). We report the total end-to-end time for each system (that means the sum of both phases for TraceProv, Muller, and SmokedDuck) and a timeout of 10 min on PostgreSQL and 5 min on DuckDB. On the right of each plot we show the distribution of the runtime of TraceProv relative to the fastest system per query.

TraceProv outperforms all other systems, often by a large margin, on both PostgreSQL and DuckDB. When computing lineage for all result rows (P_all) on average TraceProv is 2.0x (2.4x) faster than the fastest system per query on DuckDB (PostgreSQL). When only the lineage of a single result row is requested, TraceProv is still on average 1.2x (2.0x) faster on DuckDB (PostgreSQL). Note that for some TPC-H queries, SmokedDuck does not return correct lineage as it does not include any tuples required for nested subqueries to evaluate to true in the lineage. This impacts the results of queries *Q2, Q4, Q17, Q18, Q20, Q21, Q22*. Even though SmokedDuck only returns a subset of the lineage for these queries, TraceProv is still more efficient. GProM is reasonably competitive for simpler queries with aggregation and few joins. In fact, is the only system to outperform TraceProv on some TPC-H queries: for DuckDB these are *Q14* (P_all) and *Q1, Q12, Q14, Q9* (P_single). These are queries for which GProM outperforms noprov because it optimizes away the final aggregation step of the query, an optimization that we do not support yet in TraceProv. However, when considering all TPC-H queries, GProM is not competitive. For example, TraceProv

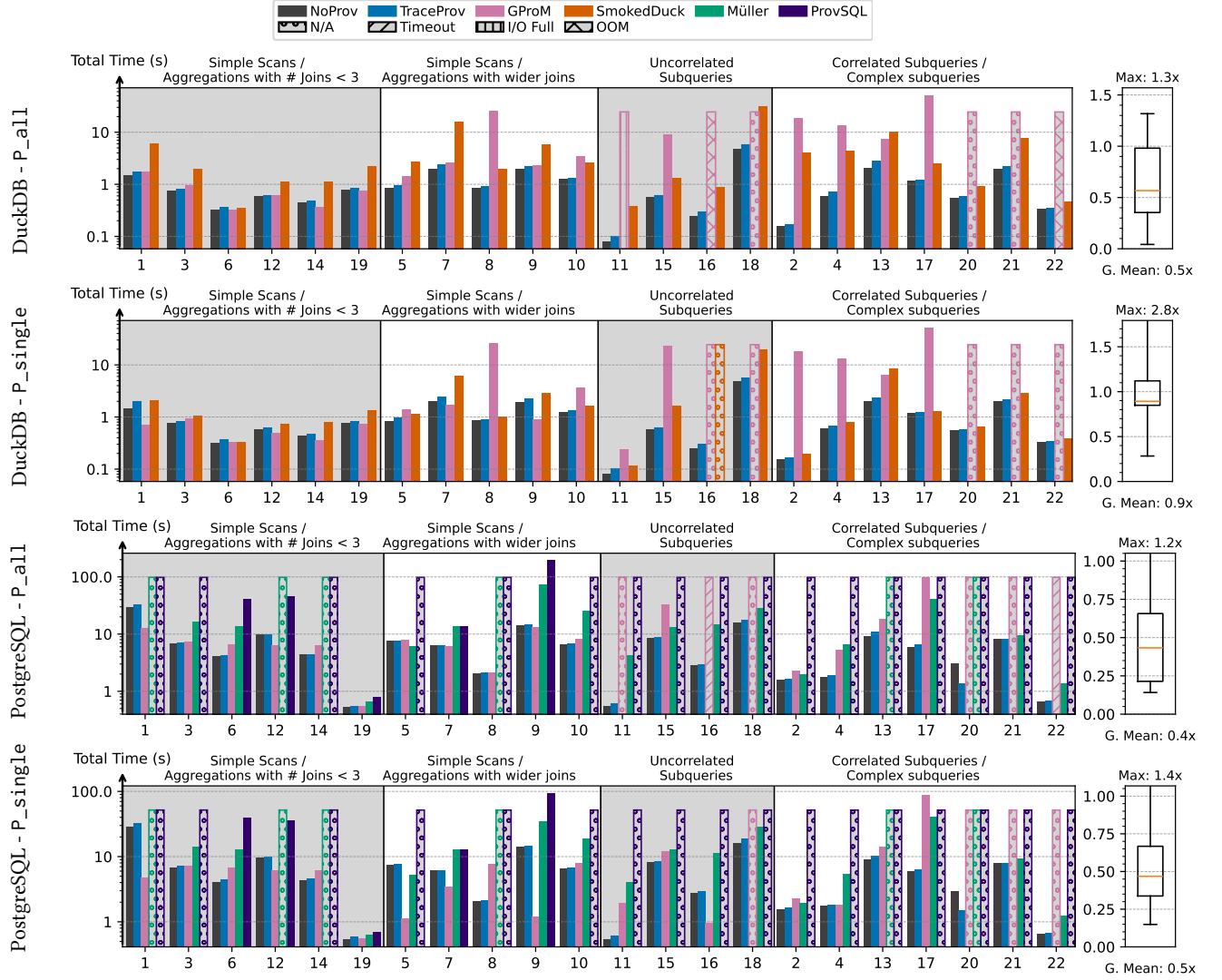
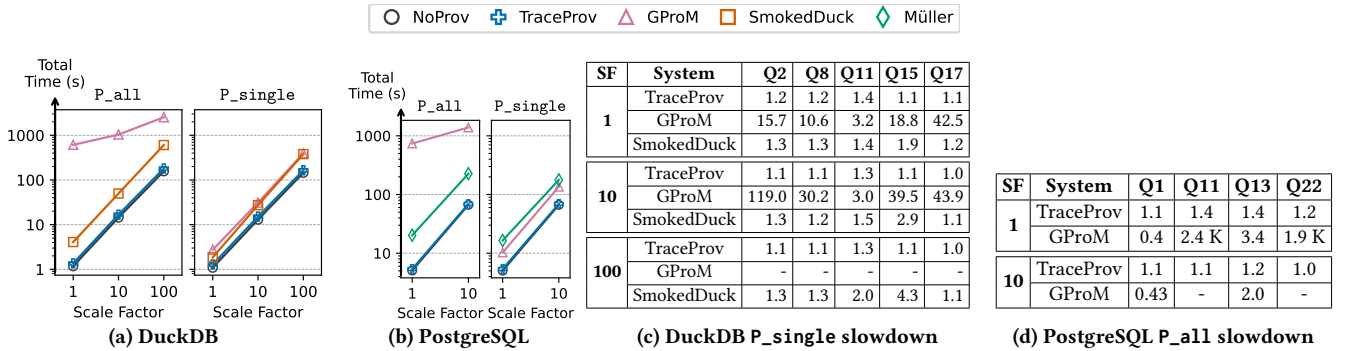
Figure 7: End-to-end runtime on TPC-H SF10: lineage of a single query result (P_{single}) and all query results (P_{all}).

Figure 8: Cumulative end-to-end time for all 22 TPC-H queries varying dataset size (SF)

outperforms GProM by several OOM for queries with multiple nested subqueries, where GProM's flat encoding enumerates all

combinations of input tuples from the provenance of each nested subquery.

TraceProv outperforms the native SmokedDuck on all queries with a factor 0.9x to 3.6x on P_single, the task for which SmokedDuck is more competitive. This is largely due to the cost SmokedDuck pays for indexing captured provenance between the first and second phase and due its lack of support for parallel query execution. As we will expand on further in Section 7.4, the cost of indexing is hard to amortize, even when lineage captured in phase one is used to serve multiple retrieval requests (executions of phase two). Note that the indexing cost was excluded in the reported measurements in [20, 21], leading to different conclusions. Muller is more competitive than GProM on queries with nested subqueries as it uses a factorized representation, but is also heavily outperformed by TraceProv (factor 0.8x to 7x on P_all) due to its high logging cost (writing a log entry entails inserting a row into a database table with an additional test to check whether the row already exists) and the disadvantage of using a bottom-up approach in the second phase which generates lineage for all intermediate results of the query even if they do not end up contributing to the final query result. This is particular obvious for top-k queries such as Q3 and Q10. Finally, ProvSQL is not competitive which is due to the fact that it captures a more expressive provenance model (provenance polynomials), the large amount of data it writes to disk, and the design choice to use a single background worker process for logging polynomial circuits that all processes involved in a query have to communicate with. Note that the system does not support several TPC-H queries as it does not support top-k queries, nested subqueries, and multi-level aggregation. For supported queries, TraceProv is faster 1.4x to 13.3x faster than ProvSQL.

Scalability in dataset size. We now evaluate how each system scales in dataset size. Figure 8a and Figure 8b show the cumulative runtime for all 22 TPC-H queries for SF=1, SF=10, and SF=100 for P_all and P_single. On all configurations, cumulative time of TraceProv is comparable to noprov cumulative time. Indeed, across all the scale factors, tasks and database configurations, the total cumulative time of TraceProv is no more than 1.19x than the cumulative time of the baseline. Increasing the scale factor exposes the limitations of competing systems. We present the slowdown of systems over noprov for queries that GProM does not scale for in Figure 8c and Figure 8d for DuckDB P_single and PostgreSQL P_all, respectively.

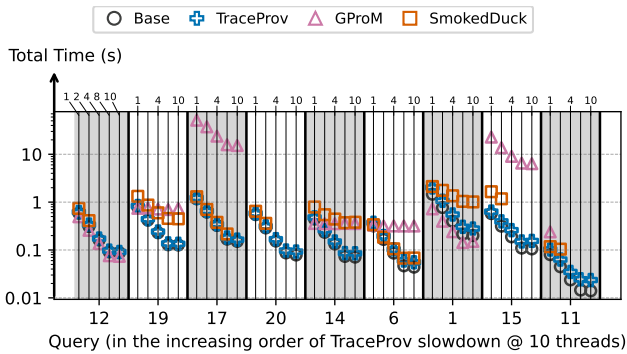


Figure 9: DuckDB TPC-H SF10 absolute query runtime varying the number of threads from 1 to 10 (shown on top).

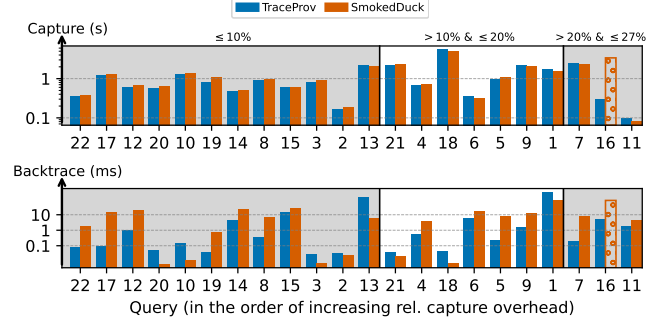


Figure 10: Logging and backtrace runtime of TraceProv and SmokedDuck for P_single on TPC-H SF10. Queries are ordered by relative capture overhead of TraceProv over noprov.

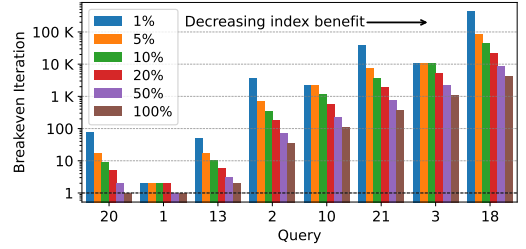


Figure 11: DuckDB - number of lineage requests (each requesting provenance for x% of query results) needed for SmokedDuck to break even with TraceProv.

7.3 Parallel Query Execution

TraceProv is designed to efficiently support parallel query engines, and we now evaluate its performance as parallelism is varied. We only consider DuckDB as it more effectively exploits available parallelism than PostgreSQL, and, thus, is more challenging. We compare against GProM and SmokedDuck on TPC-H SF=10. Recall that SmokedDuck's backtrace and index building does not utilize parallelism. In Figure 9 we vary the number of threads and compare against SmokedDuck and GProM for P_single. We only include those queries where SmokedDuck does not crash for at least one multithreaded setting. The queries are arranged in the order of increasing TraceProv slowdown (compared to the baseline) at 10 threads. TraceProv's per-query slowdown over noprov for 10 threads ranges from 1.04x to 1.68x. Further, with increasing parallelism, TraceProv's performance over competing systems significantly increases. For example, for Q19, TraceProv is 10% (86ms) slower than GProM and is 60% (494ms) faster than SmokedDuck. However, for 10 threads, TraceProv is 449% (605ms) faster than GProM, and is 232% (313ms) faster than SmokedDuck. GProM scales poorly for queries Q19, Q14, Q6. For these queries, while GProM execution time remains almost constant, TraceProv continues to scale. In [18] we show results for all 22 TPC-H queries on a newer version of DuckDB not supported by SmokedDuck. Across all settings, TraceProv's maximum slowdown over noprov was only 1.73x.

7.4 Amortization of SmokedDuck Indexing Cost

As SmokedDuck is the only native and also the most competitive in terms of overall performance, we present a more detailed comparison against this system. For completeness, we include a detailed

analysis of the TPC-H performance of the other baselines in [18]. Recall that SmokedDuck is significantly slower than TraceProv for TPC-H SF10 even for P_single due to its limitation to a single thread and due to the large cost of indexing the captured lineage after logging. As the created index structures can improve the runtime of retrieving lineage in backtrace, a reasonable question to ask is whether it is possible for this overhead to be amortized across multiple requests, e.g., a user iteratively explores the lineage of individual query result tuples resulting in a sequence of backtrace requests. To answer this question, we first show the individual runtimes for logging and backtrace for both systems and then evaluate amortization across multiple requests.

Figure 10 we show the breakdown of runtime into logging and backtrace (P_single) for TPC-H SF10. Recall that SmokedDuck does not return full lineage for some TPC-H queries as it does not return any lineage for nested subqueries. For a fair comparison, we modified TraceProv to return the same lineage as SmokedDuck for this experiment. The queries are sorted by the relative capture overhead of TraceProv compared to noprov. In terms of capture cost both systems exhibit similar performance (SmokedDuck did crash for Q16 during backtrace). This demonstrates the efficiency of TraceProv’s logging framework that is competitive with SmokedDuck, even through SmokedDuck has direct access to DuckDB’s execution engine internals. For backtrace, the differences between the systems are more pronounced. For some queries, mostly with smaller lineage, the fine-grained index structures constructed for SmokedDuck pay off while TraceProv benefits light-weight data skipping for heavier queries. Furthermore, due to its combination of propagation and logging, TraceProv benefits from having to join less logs during backtrace, e.g., SmokedDuck will create a log for each join of a query while TraceProv will create a single log.

To evaluate amortization of indexing cost, we capture lineage for one of the TPC-H queries and then repeatedly request the lineage for a randomly selected fraction of the query’s result (1% up to 100%). We report the mean number of such requests that have to be served before the two systems break even, repeating each experiment 1,000 times. Obviously, it is not possible for SmokedDuck to break even for queries for which TraceProv is faster in backtrace and, thus, these queries are excluded. The results are shown in Figure 11 where queries are sorted based decreasing index benefits. For 14 out of 22 TPC-H queries, TraceProv is faster in backtrace than SmokedDuck. For the remaining 8 queries, it is technically possible for SmokedDuck to outperform TraceProv when the later is limited to single-threaded execution. However, the number of requests (requesting the lineage of each result row 10s to 1000s of times) that are necessary to amortize indexing are unlikely to occur in practice. We present a detailed discussion of how SmokedDuck builds its indexes in [18].

7.5 Provenance Polynomials

We now evaluate capturing provenance polynomials. TraceProv supports generation of provenance polynomials for SPJ queries. On PostgreSQL, we compare against ProvSQL and GProM. On DuckDB, we compare against SmokedDuck and GProM. We require all systems to output a string representation of the polynomial for every result tuple of a query. Both ProvSQL and TraceProv

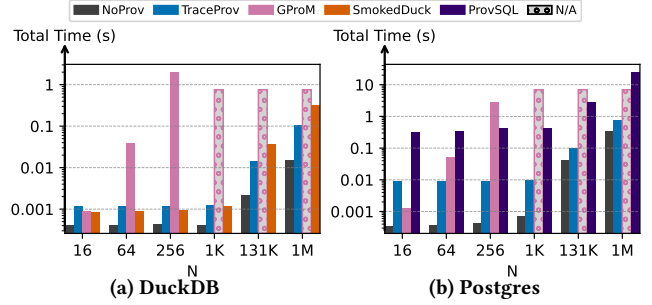


Figure 12: Runtime for capturing provenance polynomials.

factorize polynomials based on the structure of the input query. For ProvSQL at every **DISTINCT** in the query, the system sums up the polynomials attached to all intermediate results with the same values, i.e., factorization can be controlled by adding duplicate elimination at strategic places in the query.

To stress-test a systems capability to exploit factorization, we use a query template shown below that produces a single result tuple (as a has a single distinct values in all input table). Here N controls the provenance size by determining how many rows from R_i are returned.

$$\delta(\Pi_a(\sigma_{b \leq N}(R_1))) \times \delta(\Pi_a((\sigma_{b \leq N}(R_2)))) \times \delta(\Pi_a(\sigma_{b \leq N}(R_3)))$$

Let r_i denote tuples from R_1 , s_i denote tuples from R_2 and t_i denote tuples from R_3 . TraceProv, SmokedDuck and ProvSQL produce a polynomial the form $(\sum_{i=1}^N r_i) \cdot (\sum_{j=1}^N s_j) \cdot (\sum_{k=1}^N t_k)$ whose size is linear in N . GProM, however, produces a polynomial of the form $\sum_{i,j,k} (r_i \cdot s_j \cdot t_k)$ whose size is $O(N^3)$. We present the execution time for capturing polynomials in Figure 12. As GProM does not utilize factorization, it does not scale to larger values of N , running out of memory. On DuckDB (Figure 12a), SmokedDuck initial outperforms TraceProv for smaller values of N . However, as N is increased to 1M, TraceProv outperforms SmokedDuck. TraceProv consistently outperforms ProvSQL.

Table 1: Logging performance and log size per log entry

		Number of Rows							
		PostgreSQL				DuckDB			
Time μs	TraceProv	10	1K	100K	10M	10	1K	100K	10M
	TraceProv (file)	597	6.1	0.11	0.05	20	0.2	0.004	0.003
	Muller	86	5.0	4.1	4.1	27	0.29	0.006	0.003
	ProvSQL	188	5.8	2.4	2.4				
	SmokedDuck					0.8	0.009	0.001	0.001
Size Bytes	TraceProv	1 K	16	8.0	8.0	3 K	32	4.3	4.0
	Muller	819	73	68	68				
	ProvSQL	2 M	25 K	469	350				
	SmokedDuck					7.2	4.0	4.0	4.0

7.6 Performance of Logging

One of the main enablers of high capture performance of TraceProv is its efficient parallel logging framework. We now evaluate logging performance in isolation. We now compare the cost of logging as the mean time and storage size per log entry. We use a simple query $Q = \Pi_a(T)$ over relation $T(a)$ where a is a 4 byte integer.

Table 2: Backtrace optimizations: speed-up over unoptimized TraceProv for TPC-H SF10

Opt	Query																					
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
JR	0.99	0.77	2.1	1.1	1.4	0.88	1.3	1.1	1.5	2.0	1.2	1.1	1.0	0.97	1.1	1.1	1.0	1.0	2.8	2.8	1.0	1.2
Z	0.99	2.1	52	0.99	1.1	0.58	1.0	1.1	2.4	4.1	1.5	0.52	1.0	0.68	1.1	1.1	1.0	1.6	3.1	5.5	1.7	1.0
P	0.57	2.1	57	1.1	1.9	0.61	1.7	1.0	8.1	23	1.6	1.0	1.1	0.68	1.4	0.34	1.1	1.6	3.1	5.5	1.7	1.1

For this query, all rows in T belong in the provenance. We vary the number of entries logged by varying $|T| \in \{10, 1K, 100K, 10M\}$. For ProvSQL, we explicitly call the function `create_gate` to create a gate per row. The results are shown in Table 1. TraceProv has the highest per-entry time for fewer log entries (10 and 1K). However, as the number of log entries is increased, the per-log time drops significantly. At 10M log entries, TraceProv is 48 $\times$ faster than ProvSQL, the next best system on PostgreSQL. For DuckDB, TraceProv benefits from vectorization for UDFs, resulting in a $\sim 20\times$ speedup compared to PostgreSQL at 10M rows. While SmokedDuck is the fastest system in terms of raw logging performance, the absolute runtime difference per log entry at 10M rows is only 2ns. For PostgreSQL TraceProv uses 8 bytes to represent a log entry. At fewer number of log entries (10 and 1K), the storage per log entry is higher since TraceProv overrequests pages from the OS. Both Muller and ProvSQL have significantly higher storage requirements (8x and $\sim 43\times$ at 10M rows). For DuckDB, TraceProv uses 4 bytes to represent log entries. For increasing $|T|$, the system approaches the per entry size of SmokedDuck which also uses 4 byte entries. When using mmaped file files, TraceProv shows slightly higher overhead when logging few entries, but still approaches the 3ns per log entry at 10M rows.

7.7 Backtrace Optimizations

As described in the Section 5, TraceProv implements multiple optimizations for the P_single task. Table 2 shows the speed-up for lineage capture backtrace over TraceProv with all optimizations deactivated. Each row applies all the optimizations above that row. JR (join-rewrite) is the optimization that eliminates the join with the top-most log if the lineage of a single result is requested, Z utilizes TraceProv’s zone maps for data skipping, and P applies partition pruning. JR improves performance by 2x for several queries, even without use of zonemaps, at the cost of slight degradation for some queries. Activating both the join rewrites and zonemaps (Z) significantly improves the backtrace for most TPC-H queries e.g., 52x for $Q3$ and 5.5x for $Q20$). Additionally applying partitioning (P) further improves the performance for $Q9$ (8x) and $Q10$ (23x). While the unoptimized backtrace has a median time over all TPC-H queries of 2.75ms, activating all optimizations reduces this to 1.52ms. Note that applying partitioning causes DuckDB’s query optimizer to switch from *Perfect Hash Join* to *Hash Join* in some cases which reduces performance for some queries.

8 CONCLUSIONS

In this paper, we present TraceProv, a novel framework for implementing provenance support for relational databases that solely relies on extensibility mechanisms to implement provenance support without compromising performance. In fact, TraceProv’s performance is on par or exceeds SmokedDuck, the SOTA approach

for provenance capture that extends the internals of DuckDB and our system significantly outperforms prior work that uses extensibility mechanisms. This is made possible due to a combination of systems and conceptual innovations. We demonstrating the generality and flexibility of our approach by implementing in both a traditional row-based system (PostgreSQL) as well as in a column system (DuckDB).

REFERENCES

- [1] P. Agrawal, O. Benjelloun, A. D. Sarma, C. Hayworth, S. U. Nabar, T. Sugihara, and J. Widom. Trio: A system for data, uncertainty, and lineage. In *VLDB*, pages 1151–1154, 2006.
- [2] Y. Amsterdamer, D. Deutch, and V. Tannen. Provenance for aggregate queries. In *PODS*, pages 153–164, 2011.
- [3] B. Arab, S. Feng, B. Glavic, S. Lee, X. Niu, and Q. Zeng. Gprom - a swiss army knife for your provenance needs. *IEEE Data Eng. Bull.*, 41(1):51–62, 2018.
- [4] P. Buneman, S. Khanna, and W.-C. Tan. On propagation of deletions and annotations through views. In *PODS*, pages 150–158, 2002.
- [5] Y. Cui and J. Widom. Lineage tracing in a data warehousing system. In *ICDE*, pages 683–684, 2000.
- [6] K. M. Dannert, E. Grädel, M. Naaf, and V. Tannen. Semiring provenance for fixed-point logic. In C. Baier and J. Goubault-Larrecq, editors, *29th EACSL Annual Conference on Computer Science Logic, CSL 2021, January 25–28, 2021, Ljubljana, Slovenia (Virtual Conference)*, volume 183, pages 17:1–17:22, 2021.
- [7] D. Deutch, Z. Ives, T. Milo, and V. Tannen. Caravan: Provisioning for what-if analysis. *CIDR*, 2013.
- [8] B. Dietrich, T. Müller, and T. Grust. Data provenance for recursive sql queries. In A. Chapman, D. Deutch, and T. Malik, editors, *TaPP*, pages 9:1–9:8, 2022.
- [9] S. Feng, B. Glavic, and O. Kennedy. Efficient approximation of certain and possible answers for ranking and window queries over uncertain data. *Proc. VLDB Endow.*, 16(6):1346–1358, Feb. 2023.
- [10] F. Geerts, G. Karvounarakis, V. Christophides, and I. Fundulaki. Algebraic structures for capturing the provenance of sparql queries. In *ICDT*, 2012.
- [11] F. Geerts and A. Poggi. On database query languages for k-relations. *Journal of Applied Logic*, 8(2):173–185, 2010.
- [12] B. Glavic. Data provenance - origins, applications, algorithms, and models. *Foundations and Trends® in Databases*, 9(3-4):209–441, 2021.
- [13] B. Glavic and G. Alonso. Perm: Processing provenance and data on the same data model through query rewriting. In *ICDE*, pages 174–185, 2009.
- [14] B. Glavic and G. Alonso. Provenance for nested subqueries. In *EDBT*, pages 982–993, 2009.
- [15] B. Glavic, R. J. Miller, and G. Alonso. Using sql for efficient generation and querying of provenance information. In *search of elegance in the theory and practice of computation: a Festschrift in honour of Peter Buneman*, pages 291–320, 2013.
- [16] T. J. Green, G. Karvounarakis, and V. Tannen. Provenance semirings. In *PODS*, pages 31–40, 2007.
- [17] X. Hu, S. Sun, S. Patwa, D. Panigrahi, and S. Roy. Aggregated deletion propagation for counting conjunctive query answers. *PVLDB*, 14(2):228–240, 2020.
- [18] V. Jha and B. Glavic. Traceprov: All you need is log - efficient bolt-on provenance capture for complex relational queries (extended version). Technical report, UIC, 2026.
- [19] A. Meliou and D. Suciu. Tiresias: The database oracle for how-to queries. In *SIGMOD*, pages 337–348, 2012.
- [20] H. Mohammed, C. Summers, S. Kaushik, and E. Wu. Smokedduck demonstration: Sqlstepper. In S. Das, I. Pandis, K. S. Candan, and S. Amer-Yahia, editors, *SIGMOD*, pages 183–186, 2023.
- [21] H. Mohammed and E. Wu. Lineage capture trade-offs: A case study in duckdb. In *Proceedings of the ProvenanceWeek 2025*, pages 32–36, 2025.
- [22] H. Mohammed, E. Wu, A. Yao, C. Summers, L. Fokas, G. Y.-Y. Chan, S. Mitra, and H. Zhong. Fade: More than a million what-ifs per second. *PVLDB*, 18(4):943–955, 2024.
- [23] M. Mutsuzaki, M. Theobald, A. de Keijzer, J. Widom, P. Agrawal, O. Benjelloun, A. Das Sarma, R. Murthy, and T. Sugihara. Trio-one: Layering uncertainty and lineage on a conventional dbms. In *CIDR*, pages 269–274, 2007.
- [24] T. Müller, B. Dietrich, and T. Grust. You say ‘what’, i hear ‘where’ and ‘why’: (mis-) interpreting sql to derive fine-grained provenance. *PVLDB*, 11(11):1536–1549, 2018.
- [25] T. Müller and T. Grust. Provenance for sql through abstract interpretation: Value-less, but worthwhile. *PVLDB*, 8(12), 2015.
- [26] X. Niu, R. Kapoor, B. Glavic, D. Gawlick, Z. H. Liu, V. Krishnaswamy, and V. Radhakrishnan. Provenance-aware query optimization. In *ICDE*, pages 473–484, 2017.

- [27] X. Niu, R. Kapoor, B. Glavic, D. Gawlick, Z. H. Liu, V. Krishnaswamy, and V. Radhakrishnan. Heuristic and cost-based optimization for diverse provenance tasks. *TKDE*, 2018.
- [28] X. Niu, Z. Liu, P. Li, B. Glavic, D. Gawlick, V. Krishnaswamy, Z. H. Liu, and D. Porobic. Provenance-based data skipping. *PVLDB*, 15(3):451–464, 2021.
- [29] D. Olteanu. Recent increments in incremental view maintenance. In F. Geerts and W. Martens, editors, *PODS*, pages 8–17, 2024.
- [30] P. Pintor, R. L. de C. Costa, and J. M. Moreira. A dbms-independent approach for capturing provenance polynomials through query rewriting. *IEEE Access*, 14:66114–66133, 2026.
- [31] F. Psallidas and E. Wu. Smoke: Fine-grained lineage at interactive speed. *PVLDB*, 11(6):719–732, 2018.
- [32] A. Sen, S. Maniu, and P. Senellart. Provsq: A general system for keeping track of the provenance and probability of data. In *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. IEEE, 2026.
- [33] P. Senellart, L. Jachiet, S. Maniu, and Y. Ramusat. Provsq: provenance and probability management in postgresql. *PVLDB*, 11(12):2034–2037, 2018.
- [34] G. Van den Broeck and D. Suciu. Query processing on probabilistic data: A survey. 2017.